

Software Evolution Analysis Report

Inventory Management System

By

Bruno Nicholas Sserunkuma (SEP15U01175)

1. Introduction

The Inventory Management System (IMS) is a full-stack web application developed to help businesses track, manage, and optimize their stock levels in real time.

Built using a PHP/Node.js backend, a PostgreSQL relational database, and a responsive front-end dashboard, the system centralizes product cataloging, stock intake and dispatch, supplier management, and automated low-stock alerting into a single platform.

The project was developed collaboratively by a four-person team and hosted on GitHub, which served as the primary platform for version control, issue tracking, and release management. The system targets small to medium-sized enterprises, such as retail outlets, warehouses, and distribution centers, that require a reliable, audit-able record of inventory movements without the overhead of expensive enterprise resource planning software.

Over the course of development and post-release maintenance, GitHub's built-in features were used extensively to coordinate work across the team, ensure code quality, manage evolving requirements, and maintain a clear history of all changes made to the code-base. This report examines how those features were leveraged and how they supported the ongoing evolution of the software.

2. GitHub Features Used

2.1 Repositories

The project was hosted in a single GitHub repository under the organization account `inventory-system-org`. The repository served as the canonical source of truth for all source code, configuration files, database migration scripts, and documentation.

Repository settings were configured to enforce branch protection on the main branch, requiring at least one approving review before any pull request could be merged. Read, write, and admin permissions were assigned to team members according to their roles, ensuring that junior developers could contribute freely while senior engineers retained merge authority.

2.2 Commits

The team adopted a structured commit message convention following the Conventional Commits specification, for example, fix: prevent negative stock on concurrent orders or feat: add bar-code scanner support. This discipline meant the commit history served as a readable change log and made it straightforward to identify the origin of any regression using git bisect. Over the project life-cycle, over 340 commits were recorded, each scoped to a single logical change to keep diffs reviewable and rollbacks surgical.

2.3 Branches

A Git-flow-inspired branching strategy was adopted. The main branch held only production-ready code. A develop branch served as the integration target for all feature and fix work. Individual branches, named according to the pattern feature/bar code-scanner or fix/negative-stock-bug, were created for each unit of work and deleted after merging. This approach kept the main branch stable, enabled parallel development across the team without interference, and made it easy to cherry-pick critical hot-fixes directly to production when necessary.

2.4 Pull Requests

Every change to the develop or main branch was gated behind a pull request (PR). PRs included a description of the change, a link to the related issue, and a testing checklist. Reviewers used GitHub's inline comment feature to discuss specific lines of code, request changes, or approve. Automated checks, including linting, unit test runs, and a security audit via the package manager audit, were triggered on every PR using GitHub Actions, so that no code with failing tests or known vulnerabilities could be merged. Over the project's history, 87 pull requests were opened, of which 83 were merged and 4 closed without merging after the related work was superseded.

2.5 Issues

GitHub Issues was used as the project's primary task and bug tracker. Issues were categorized with labels including bug, feature, refactor, and documentation, and assigned priority tags (high, medium, low). Each issue contained a clear description, reproduction steps where relevant, and was linked to the PR that resolved it using closing keywords such as Closes #23. This created a traceable connection between reported problems and their solutions, which proved invaluable during retrospectives and when on-boarding a new team member mid-project.

2.6 Milestones

Three milestones were defined to:

- structure delivery: v1.0,
- Core Inventory Features, v1.1,
- Reporting and Alerts, and v1.2,

This is tallying with Access Control and Security Hardening.

Each milestone grouped the issues required to reach that release target, with a due date attached.

Milestone progress was visible as a percentage bar on GitHub, giving the team and stakeholders an at-a-glance view of delivery status. Milestones also helped prioritize backlog triage, any issue not assigned to an active milestone was treated as a candidate for deferral.

3. Software Evolution Support

Lehman's Laws of Software Evolution describe how software systems must continuously adapt to remain useful. GitHub directly supports the four canonical maintenance categories that drive this evolution.

3.1 Corrective Maintenance

Corrective maintenance addresses defects discovered after release. GitHub Issues provided the mechanism for users and testers to report bugs with sufficient detail for developers to reproduce and fix them. For instance, issue #12, stock quantity going negative during concurrent order processing, was reported, triaged as high priority, assigned to a developer within the same day, and resolved through a targeted hotfix branch. The PR for the fix was reviewed and merged within 24 hours, and the milestone tracker was updated accordingly. The closed issue then formed part of the permanent audit trail, documenting what was broken, who fixed it, and how.

3.2 Adaptive Maintenance

Adaptive maintenance involves modifying software to keep it functional as its external environment changes, such as updated dependencies, changes in operating systems, or evolving API contracts. When backend framework used released a breaking change in a core crypto module, a new issue was raised and a feature branch was used to update the affected code in isolation.

GitHub's Dependabot integration also raised automated pull requests when outdated packages were detected in package.json, enabling the team to review and apply dependency updates incrementally rather than allowing technical debt to accumulate. Each adaptive change was tied to a milestone, ensuring it did not delay planned feature work.

3.3 Perfective Maintenance

Perfective maintenance improves performance, usability, or functionality beyond what was originally specified. The IMS roadmap was driven largely by user feedback collected through GitHub Discussions and converted into feature issues. Adding barcode scanner support (issue #31), introducing a stock turnover chart (issue #38), and implementing role-based access control (issue #44) were all examples of perfective

maintenance driven by real user demand. The milestone structure ensured these enhancements were batched into coherent releases rather than shipped piecemeal, giving users a predictable upgrade cadence and reducing regression risk.

3.4 Preventive Maintenance

Preventive maintenance reduces the risk of future failures by improving the internal structure of the code without changing its observable behavior.

Refactoring tasks such as splitting the oversized *ProductController* into discrete services (issue #57) and increasing unit test coverage to 80% (issue #63) were tracked as first-class GitHub issues, planned into milestones, and reviewed through pull requests just like any feature work.

GitHub Actions enforced that test coverage could not regress, any PR that lowered coverage below the threshold failed the automated check and was blocked from merging. This made preventive work a visible, measurable activity rather than an afterthought.

4. Challenges Encountered

4.1 Merge Conflicts

The most frequent challenge was merge conflicts, particularly in high-activity files such as *package-lock.json* and the database migration index. When multiple developers worked on related areas simultaneously and submitted PRs in quick succession, resolving conflicts consumed significant time and introduced the risk of inadvertently discarding another developer's changes. The team addressed this by enforcing a policy of rebasing feature branches onto develop before opening a PR, and by reducing the lifespan of branches, aiming to merge within 48 hours of opening to minimize divergence.

4.2 Inconsistent Issue Descriptions

Early in the project, issues were filed with inconsistent levels of detail, some lacked reproduction steps, expected versus actual behavior, or environment information. This slowed down triage and led to back-and-forth comments before a developer could begin work. The team resolved this by introducing GitHub Issue Templates, separate templates for bugs, feature requests, and refactoring tasks, which prompted reporters to fill in structured fields. Issue quality improved measurably within two weeks of introducing the templates.

4.3 CI Pipeline Failures Blocking Progress

On two occasions, a flawed GitHub Actions workflow configuration caused the CI pipeline to fail for all open PRs simultaneously, once due to a misconfigured environment variable in the test runner, and once due to an expired token for a third-party service used in integration tests. Because branch protection required passing CI checks before merging, this effectively blocked the entire team. The team responded by documenting a run-book for common CI failures, establishing a secondary reviewer role responsible for monitoring pipeline health, and separating integration tests into an optional workflow so that unit tests could still gate merges even when the external service was unavailable.

4.4 Scope Creep in Pull Requests

Several pull requests grew well beyond their original scope as developers noticed adjacent issues while working on a feature. Large PRs, some exceeding 800 changed lines, were difficult to review thoroughly and frequently sat open for several days while reviewers found time. The team introduced a soft limit of 400 lines per PR and a norm of splitting pre-existing issues discovered during a feature branch into separate issues rather than fixing them in-place. Smaller PRs moved through review faster and with fewer post-merge defects.

5. Reflection

Working on the Inventory Management System across multiple months and a four-person team made it clear that version control tooling is not merely a convenience, it is the infrastructure on which collaborative software evolution depends. GitHub provided not just a place to store code, but an integrated environment for communication, quality assurance, planning, and historical record-keeping.

One of the most important lessons learned was the value of treating non-functional work, refactoring, documentation, and test coverage, with the same discipline as feature development. Before adopting GitHub Issues and milestones for this work, it was easy for preventive and corrective maintenance to be deferred indefinitely in favour of visible new features. Once these tasks became trackable issues with labels, assignees, and milestone membership, they competed for attention on equal footing and were completed consistently.

The pull request review process, while occasionally a bottleneck, proved to be the single highest-value practice in the team's workflow. The discipline of writing a clear PR description, linking to an issue, running automated checks, and requesting a human review caught defects that would otherwise have reached production. It also served as a continuous knowledge-sharing mechanism, developers regularly learned about parts of the codebase they had not written by reviewing others' code, which reduced silo-ed knowledge and increased the team's collective ownership of the system.

GitHub also reinforced an important principle of software evolution: that change is not a sign of poor initial design, but an inevitable property of any living system. The commit history, issue tracker, and milestone records together form a narrative of how the Inventory Management System grew in response to real-world use, bugs encountered by actual users, performance concerns raised under load, and features requested as business needs evolved. This narrative has practical value: when a new team member joins, they can trace why the system is the way it is, not just what it does.

Looking ahead, the team intends to expand the use of GitHub Projects for sprint planning, integrate automated release notes via GitHub Releases, and adopt semantic versioning tied directly to milestone completions. These additions will further tighten the loop between planning, development, and delivery, supporting the ongoing evolution of the IMS with greater transparency and predictability.

By, Bruno Nicholas Sserunkuma

Reg: **SEP15U01175**
